Software defence against RAM-replay attacks on TME-only commodity hardware: tagged-pointer MACs in BEAM

Claude Opus 4.7[1m]
Anthropic

Sam Williams
sam@arweave.org

April 2026 — design note (companion to LapEE)

Abstract

Intel Total Memory Encryption (TME) provides confidentiality for DRAM contents but not freshness: a physical-access attacker can capture ciphertext at a given physical address and replay it later, and the CPU will decrypt the captured value identically. The proper defence is hardware memory-integrity trees (SEV-SNP RMP, TDX MAC trees), which are unavailable on the commodity laptop hardware that the LapEE project targets. We describe a software-only defence that detects RAM replay against a running BEAM virtual machine by storing a per-pointer truncated MAC of the pointed-to bytes in the unused high bits of every term-bearing pointer. Because BEAM terms form a graph rooted in a small set of process-control structures, an interior replay is detected on the next dereference of the parent pointer, and the attacker's effective surface collapses to graph roots. Combined with keeping process-control blocks resident in CPU cache, the residual surface is reduced to a handful of mostly-static pages. We sketch the construction, its bit budget, the engineering and performance trade-offs, and the unresolved design questions, and conclude with a path to a measured prototype.

1 Threat model

Adversary. A physical-access attacker who can interpose on the DRAM channel of a TME-protected node — via a memory bus tap, a malicious DMA peripheral, an FPGA interposer between the CPU package and DIMMs, or a sufficiently capable cold-boot extraction. The attacker reads and writes ciphertext at physical addresses of their choosing during the execution of a target program.

Goal. Influence the output of an in-flight computation on a LapEE-attested HyperBEAM node by making it process stale data without detection. Concretely: rewind a memory location holding a request parameter, an authorisation token, a counter, or any term-graph node participating in an attested workload, and have the node sign or commit to the resulting (now-incorrect) computation as if nothing happened.

Out of scope. Replay against the TPM itself — LapEE already defends against this with per-attestation nonces signed inside the TPM2 quote. Replay against the AK private key — the AK lives in the TPM and is never exposed to RAM. The class of attack we are defending here is replay against the *computation* happening in BEAM userspace, where the TPM has already certified an honest boot but a runtime memory adversary subverts ongoing work.

2 Why TME alone is insufficient

TME on Intel client silicon is constructed as AES-XTS over the DRAM bus with a per-boot key and a tweak derived from the physical address. The construction has two properties relevant here:

— *Same address, same key, same ciphertext, same plaintext.* The encryption is deterministic; ciphertext at address P at time t_1 decrypts identically to ciphertext at P at time t_2 as long as the key has not rotated. The key only rotates on power cycle.

— *No integrity tag.* TME stores no MAC, no version counter, no Merkle commitment. The CPU has no way to detect that the ciphertext at P was substituted for an older ciphertext at the same P.

The freshness gap is exactly what AMD SEV-SNP's reverse-map table (RMP) and Intel TDX's memory-encryption MAC trees were designed to close. They are not yet available on the commodity laptop class targeted by LapEE (Framework 13 with Intel 12th/13th-gen Core silicon ships TME but not TDX; AMD Ryzen variants ship SME but not SEV-SNP for client SKUs).

In the absence of hardware integrity, BEAM's natural memory churn — generational copying GC, per-request ephemeral processes, per-scheduler allocator separation — already imposes some practical difficulty on a replay attacker, because the window in which a captured ciphertext remains valid for a given logical object is bounded by the next minor GC cycle (typically tens of milliseconds under load). We have proposed amplifying this churn through configuration tuning. But this is a probabilistic defence: with enough profiling and timing, a sufficiently patient attacker can succeed. The defence we describe here is intended to make the attack *detectable* in addition to *difficult*.

3 Construction

3.1 Tagged pointers in BEAM

BEAM represents Erlang terms in single machine words. The bottom 2–4 bits encode a major term tag (immediate, list cell, boxed, etc.); for boxed and list-cell terms the remaining bits hold a pointer to a heap allocation. On x86_64 Linux, virtual addresses use at most 47 bits in canonical form, so a term-bearing pointer naturally has ~ 16 unused high bits. On a laptop addressing ≤ 32 GB of RAM the live address space is much smaller — 35–36 bits of address suffice — which leaves ~ 24 high bits genuinely free.

3.2 Per-pointer integrity tag

We propose to use those high bits as a truncated message authentication code (MAC) over the bytes that the pointer points at:

$$tag(p) = \mathrm{MAC}_K(content(p))[0..n)$$

where K is a per-boot key held in a CPU register that is never spilled to memory (a BEAM scheduler's reserved XMM register is a natural choice), and n is the integrity-bit budget — 24 bits for a 36-bit address space, 28 bits if we accept a 32-bit address cap.

The MAC is computed at every site that creates a term-bearing pointer: allocation of a new boxed term, GC copy of a live term to a new arena, construction of a new tuple element. It is verified at every site that dereferences such a pointer.

A mismatch indicates that the bytes at the pointed-to address differ from the bytes that were there when the pointer was created — which is exactly the signature of a RAM replay against an interior heap location.

3.3 Bit budget

For a 64-bit machine word with BEAM's canonical 4-bit major tag:

Field	**Bits**
Major term tag (immediate / list / boxed / ...)	4
Address bits (36-bit physical address space, $\leq$ 64 GB)	36
Integrity tag (MAC)	**24**
Total	64

A 24-bit MAC is short by cryptographic standards. The relevant security question is not "can the attacker produce a forgery given an oracle" but "can the attacker, in the time window between memory capture and memory replay, find a stale ciphertext whose 24-bit MAC happens to match the new MAC stored in the live pointer". With a per-boot key and content that changes at every minor GC (≤ 100 ms), an attacker must succeed within that window. The forgery probability per attempt is 2^{-24}; against a node serving even 100 requests per second, the expected number of attempts before a successful blind forgery is $\sim 10^7$, which translates to many minutes of focused attack on a moving target. In practice the attacker is also constrained by their DRAM-tap throughput and by the necessity of guessing which captured ciphertext to replay; the effective security is materially higher than the bit count alone suggests.

3.4 MAC algorithm

The choice of MAC is a performance–security trade-off, not a cryptographic freshness question (the freshness comes from the per-boot key plus the content-hash structure, not from MAC strength). Candidates:

— **CRC-32C**: ~ 1 cycle per cache line on modern CPUs with the SSE 4.2 `crc32` instruction. Not a MAC: an attacker who can profile finds collisions deterministically. *Insufficient.*

— **HMAC-SHA-256**: cryptographically strong but $\sim$ hundreds of cycles per evaluation. *Unaffordable on the dereference hot path.*

— **Truncated AES-128 PRF**: one AES round with AES-NI is ~ 4 cycles; a full encryption is $\sim$ 25–40 cycles. Truncated to 24 bits, this composes as a near-ideal random function under standard PRF assumptions. *Likely the right choice.*

— **SipHash-2-4**: $\sim$ 10–20 cycles, designed for short keyed hashes, software-only. Reasonable fallback if AES-NI is for some reason unavailable.

The per-boot key K is derived at startup from a high-entropy source (`getrandom(2)` or `RDSEED`), placed in an XMM register reserved by the BEAM scheduler dispatch loop, and never written to memory. A check-failure trap clears the register before any debug output is emitted.

4 The graph-rooting argument

The defence's elegance comes from the structure of BEAM's term graph. Every term-bearing pointer is reachable from a small set of roots:

— **Process control blocks (PCBs)**, one per Erlang process. A PCB holds the root pointer to that process's heap, mailbox queue, register save state, and scheduler queues.
— **ETS table headers**, one per ETS table. The header holds the root of the table's internal trees / hash tables.
— **The atom table**, a single global structure mapping atom identifiers to atom-name strings.
— **Module code area**, a static area holding loaded BEAM bytecode.
— **Registered process / port / monitor / link tables**, small global tables.

If the attacker replays an *interior* term — a tuple element, a list cell, a refc-binary header — the parent pointer's MAC was computed against the term's pre-replay bytes. The next dereference of that parent pointer recomputes the MAC against the post-replay bytes and detects the mismatch. Recursively, the attacker would have to replay every ancestor up to the root; but each ancestor is itself reached through a parent pointer carrying a MAC of its own pre-replay bytes. The cascade terminates only at a root that has no parent pointer.

This means the attack surface for *undetectable* replay collapses from "every byte of the BEAM heap" to "the root tables listed above". The roots are small (PCB $\approx$ 512 bytes; ETS table header $\approx$ 256 bytes; atom table $\approx$ tens of KB), bounded in count, and accessed frequently by the scheduler.

5 Keeping roots in cache

The graph-rooting analysis still permits replay against any of the root structures themselves, because they are not pointed at by a parent. To close that gap we need the roots' bytes to never reach DRAM in the first place.

The proposed mechanism is to keep the root structures resident in on-die cache (L1 or L2) for the lifetime of the BEAM scheduler that owns them, so that the attacker's bus tap on the DRAM channel never observes the ciphertext. Linux does not expose direct cache pinning to userspace, but two practical mechanisms approximate it:

— **Access-pattern pinning.** The scheduler dispatch loop already touches its current PCB on every reduction tick; ensuring that the *root pointer* fields of all schedulable PCBs are touched at the same cadence keeps them in the L1/L2 LRU set. A small "hot-PCB-header" restructure — placing the root pointer + integrity key into a single cache line per PCB, and prefetching that cache line on every scheduler-loop iteration — makes this discipline cheap and explicit.
— **Intel CAT (Cache Allocation Technology).** Where available (Xeon-class silicon mostly), CAT supports reserving a portion of the last-level cache for a specific process. Less applicable to commodity laptop hardware, but worth noting.

With root structures held in cache and interior structures protected by tagged-pointer MACs, the residual attack surface is the small set of *cold* root structures that fall out of cache under load (e.g. PCBs of dormant processes). For these, a periodic re-MAC-and-verify pass against a separate per-table MAC fingerprint is feasible at coarse granularity (tens of milliseconds), which is sufficient given the attacker's tap-throughput constraints.

6 Engineering considerations

6.1 Performance

Every term dereference adds: a mask-off of the high bits, a MAC recomputation against the dereferenced bytes, a comparison, a conditional branch. On the BeamAsm JIT path with AES-NI, this can be implemented as four instructions plus a fault on mismatch; on the interpreter path it is more invasive. Ballpark estimates from analogous CHERI / MTE work suggest a 10–30% throughput cost is realistic for a careful implementation, with the high end seen on list-traversal-heavy benchmarks. A naïve port to the interpreter without JIT specialisation can degrade by 2×. Real numbers require measurement on a prototype.

6.2 Garbage collection

BEAM's generational copying GC moves live terms during minor and major collections. Each move invalidates the integrity tag of every parent pointer to the moved object, because the bytes are now at a different address (and, depending on the MAC variant, the MAC may also depend on the address). Two design choices:

— *MAC over content bytes only.* MACs survive GC moves; the GC need not recompute them. Cost: equal-content terms are indistinguishable, so an attacker can replay an old `{ok, 42}` over a new `{ok, 42}`, and the MAC matches. For many-equal-shaped-terms workloads (counters, state machines) this is a genuine residual surface.
— *MAC over content + address.* MACs are GC-recomputed but the attacker cannot replay even shape-identical terms. Cost: GC work increases significantly, especially for major collections that move many objects.

A reasonable hybrid is content-only MACs for immutable / shared terms (atoms, refc binaries, code) and content+address MACs for process-local mutable terms. This splits the GC cost from the security benefit appropriately, but it adds protocol complexity that needs careful validation.

6.3 Reaction policy

When a MAC mismatch is detected, the system has options:
— **Immediate halt**, kernel panic with an attestation-signed fault note.
— **Reboot**, with the next attestation envelope reporting the fault digest in PCR 15.
— **Crash the affected Erlang process only**, supervised restart, log the fault in the runtime event log so that the next attestation envelope carries it. This option is especially attractive for LapEE because the fault digest itself becomes a cryptographic record of the attack: a verifier sees an envelope claiming `memory-replay-detected` with a hash binding the fault to the attesting node's identity. That is a much more useful signal than a silent reboot.

The third option preserves the attestation channel through the attack, which is exactly what a remote verifier wants.

6.4 Compatibility with vanilla OTP

The patch changes BEAM's pointer representation; modules compiled against vanilla BEAM and loaded into a hardened runtime must either be recompiled (the safer route) or the runtime must transparently handle "untagged" pointers via a compatibility mode. We recommend the recompile route for LapEE-only deployment, accepting that a hardened LapEE node runs only LapEE-compiled BEAM bytecode.

6.5 Estimated cost of a production-quality patch

Two to four engineer-months for the core BEAM modifications, plus a comparable amount for measurement, fuzzing, and OTP regression testing. A research prototype demonstrating detection on synthetic replay against process heaps (no GC integration, no JIT specialisation, interpreter-only) is on the order of two to three weeks for someone fluent in BEAM internals.

7 Open design questions

1. *Address layout.* What address-bit cap is acceptable? 36 bits gives 64 GB and 24-bit MAC; 32 bits gives 4 GB and 28-bit MAC. The 32-bit cap is restrictive for general use but acceptable for LapEE laptops if we declare 4 GB enough for an attestation appliance.
2. *MAC scope.* Per-pointer MAC of pointed-to-bytes-only, or of bytes-plus-address, or a hybrid as above? This requires benchmarking against representative HyperBEAM workloads (HTTP request handling, attestation, ETS-heavy code).
3. *Atom table protection.* The atom table is a single global structure that grows monotonically. Protecting it with the same per-pointer scheme is overkill; a periodic Merkle root computed over the atom-name-to-id map and held in a register would be more efficient.
4. *Code area protection.* Loaded BEAM code is essentially read-only after load. A simple per-module MAC computed at load time and verified periodically is sufficient; this is independent of the per-pointer scheme.
5. *Multi-scheduler coordination.* Each scheduler's reserved register holds the per-boot key. Are there cross-scheduler dereferences (e.g. reading a remote PCB) that need a shared key view, and how is that shared without leaking K to memory?

8 Path to a prototype

We propose the following sequence:
1. **Vendored OTP fork.** Branch the OTP release matching the deployed HyperBEAM (`27.3.4`). Add a build-time configure flag `--enable-tagged-mac-replay-defence`.

2. **Minimum viable scope.** Implement per-pointer MAC for boxed terms in the process heap only. No atom-table, ETS, or code-area coverage in the prototype. Use truncated AES-128 with the per-boot key in an XMM register.
3. **Single dereference path.** Modify `HALLOC` and the boxed-pointer-deref macros in the interpreter; skip BeamAsm. This gives an honest baseline without micro-optimisation.
4. **Synthetic replay harness.** Build a test that allocates known terms, captures their bytes, then writes the captured bytes back to the same heap address while a parent process attempts to dereference. Verify the mismatch is detected.
5. **Measurement.** Run the OTP benchmark suite plus a representative HyperBEAM workload (`hb_http` request flow with `dev_tpm2 /attestation` calls) and report throughput, GC pause, and tail latency vs. vanilla OTP.
6. **Decision point.** On the basis of measured numbers, decide whether to proceed with full BeamAsm specialisation, GC integration, and root-structure caching, or to publish the prototype as a research note and wait for hardware integrity to become available on the target platform.

9 Conclusion

The construction is not novel in its ingredients — tagged-pointer integrity has appeared in CHERI, ARM MTE, SoftBound, and several research systems — but its application to BEAM specifically, in service of a software-only defence against TME-replay attacks on commodity laptop hardware, appears to be unaddressed in the literature. The graph-rooting analysis is what makes the construction interesting: the attack surface reduces to a small bounded set of roots, and keeping those roots cache-resident closes the residual gap to a practically infeasible target. The performance cost is real but likely acceptable for a node whose primary workload is HTTP request handling at ordinary rates rather than CPU-bound computation.

For LapEE today this is future work. The current build hardens what it can with built-in address-space and allocator randomisation; the tagged-MAC patch is documented here as the proper next-generation defence and as a candidate research contribution in its own right.